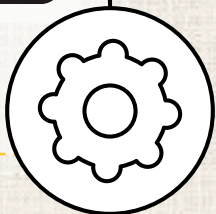


04 강

순차 구조

첨단공학부
정재화 교수



목차



1. 구조적 프로그래밍의 이해

- ✓ 명령형 프로그래밍 패러다임
- ✓ 구조적 프로그래밍 패러다임

2. 순차 구조

- ✓ 순차 구조의 개념
- ✓ 사용자 입력
- ✓ 데이터 타입 변환



학습하기

1

구조적 프로그래밍의 이해

명령형 프로그래밍 패러다임

- 1950~60년대 확립: Algol, Fortran
- 컴퓨터에게 행동을 순서대로 지시하는 방식
 - ✓ 프로그램 = 상태(state)를 가지는 시스템
 - ✓ 코드는 그 상태를 단계적으로 바꾸는 명령들의 연속
 - 할당문이 상태 변화를 일으키는 기본 수단
 - ✓ 프로그램은 명령문 실행으로 초기 → 목표 상태로 전이
- goto가 실행 흐름에 대한 통제의 어려움 가중
 - ✓ 모든 프로그램은 goto가 없는 프로그램으로 표현 가능
 - ✓ 절차적, 구조적 프로그래밍 패러다임으로 진화

스파게티 코드의 문제

```
1 i = 0
2
3 label_start:
4     if i > 5 goto label_end
5     print(f"현재 숫자: {i}")
6
7     if i == 3 goto label_skip
8
9     i = i + 1
10    goto label_start
11
12 label_skip:
13     print("3은 건너뛴니다!")
14     i = i + 1
15     goto label_start
16
17 label_end:
18     print("프로그램 종료")
```

제품 생산 공정



I 구조적 프로그래밍 패러다임

■ 명령형 프로그래밍 패러다임의 하위 개념

- ✓ goto 문을 사용하지 않고 프로그램을
3가지 제어 흐름만으로 구성하는 프로그래밍 패러다임
 - 순차(sequence) 구조
 - 선택(selection) 구조
 - 반복(iteration) 구조
- ✓ 제어 흐름을 블록 단위로 계층화하고 지역화

■ 프로그램 실행 흐름이 간결하고 작은 규모로 조직화가 용이

- ✓ 프로그램의 예측 가능성과 검증 가능성을 보장

goto 문이 없는 코드

```
1 i = 0
2
3 label_start:
4     if i > 5 goto label_end
5     print(f"현재 숫자: {i}")
6
7     if i == 3 goto label_skip
8
9     i = i + 1
10    goto label_start
11
12 label_skip:
13    print("3은 건너뛸니다!")
14    i = i + 1
15    goto label_start
16
17 label_end:
18    print("프로그램 종료")
```


goto 문이 없는 코드

```
1 for i in range(7):  
2     if i > 5:  
3         break  
4  
5     print(f"현재 숫자: {i}")  
6  
7     if i == 3:  
8         print("3은 건너뛰니다!")  
9         continue  
10  
11 print("프로그램 종료")
```

학습하기

2

순차 구조

순차 구조의 개념

■ 실행의 흐름을 주어지는 명령의 위치적 흐름에 따라 수행하는 구조

- ✓ 명령 라인 위에서 아래로 흐르는 직관적인 구조
- ✓ 사람이 이해하고 디버깅하기 용이



원의 넓이와 둘레 계산 프로그램 개선 3

■ 사용자로부터 반지름을 입력받고
넓이와 둘레를 출력?

```
1 amount = 12618985
2 interestRate = 0.013
3 print(f"원금: {amount:,}")
4 print(f"이율: {interestRate:%}")
5 print(f"이자는 {amount *
    interestRate:>10,.1f} 입니다.")
```

원의 넓이와 둘레 계산 프로그램 개선 3

■ 사용자로부터 반지름을 입력받고 넓이와 둘레를 출력?

```
1 # 원의 넓이와 둘레 계산 프로그램
2 #반지름 할당
3 rad = 40
4 #넓이 출력
5 print(f"원의 넓이는 {rad**2 * 3.141:.1f} 입니다.")
6 #둘레 출력
7 print(f"원의 둘레는 {2 * rad * 3.141:.1f} 입니다.")
```

사용자 입력

input () 함수

- ✓ 사용자로부터 데이터를 입력받는 내장 함수
- ✓ 키보드 입력을 한 줄의 문자열로 읽어 반환
- ✓ input 함수의 파라미터는
입력을 안내하기 위한 프롬프트 역할
 - 예: "반지름을 입력하세요: "

```
rad = input('반지름을 입력하세요: ')
-----
```

입력 대기 후 입력값 반환

기호 삼각 모형으로 출력하기

■ 사용자가 입력한 기호를 삼각 모형으로 출력하는 프로그램을 작성하시오.

기호를 입력하세요: * ↵

*

초를 분과 초로 변환하기

- 사용자로부터 입력 받은 '초' 단위의 시간을 몇 분 몇 초인지 계산하여 출력하는 프로그램을 작성하시오.

시간을 입력하세요(초단위): 200 ↵
입력하신 시간은 3분 20초 입니다.

프로그램의 오류

❏ 설계 결함, 논리적 또는 문법 오류로 프로그램이 의도한 대로 결과를 생성하지 못하는 상황

✓ 구문 오류(syntax error)

- 문법적으로 적합하지 않는 명령문 입력 시 발생
- 프로그램이 실행되기 전, 인터프리터/컴파일러가 탐지

✓ 실행 오류(runtime error)

- 실행 시작은 되지만 실행 중에 진행 불가 상황 발생으로 프로그램이 중단되는 오류

✓ 의미 오류(semantic error)

- 프로그래머의 의도와 다른 잘못된 결과를 생성
- 논리적/의미적으로 잘못 해석되는 명령문 작성 시 발생

데이터 타입 변환

- 한 데이터를 다른 데이터 타입의 데이터로 변환
- 데이터 타입 변환 내장 함수
 - ✓ 문자열 타입으로 변환: `str()`
 - ✓ 정수 타입으로 변환: `int()`
 - ✓ 소수 타입으로 변환: `float()`

```
1 rad = int("40")
```

```
1 rad = 40
```

I 데이터 타입 변환

- 한 데이터를 다른 데이터 타입의 데이터로 변환
- 데이터 타입 변환 내장 함수
 - ✓ 문자열 타입으로 변환: `str()`
 - ✓ 정수 타입으로 변환: `int()`
 - ✓ 소수 타입으로 변환: `float()`

```
1 rad = 40
```

```
1 rad = int(input("반지름을 입력하세요"))  
2 rad = int(rad)
```

데이터 타입 변환

- 한 데이터를 다른 데이터 타입의 데이터로 변환
- 데이터 타입 변환 내장 함수
 - ✓ 문자열 타입으로 변환: `str()`
 - ✓ 정수 타입으로 변환: `int()`
 - ✓ 소수 타입으로 변환: `float()`

```
1 rad = 40
```

```
1 rad = "40"
```

```
2 rad = int(rad)
```

원의 넓이와 둘레 계산 프로그램 개선 4

■ 사용자로부터 반지름을 입력받고 넓이와 둘레를 출력?

```
1 # 원의 넓이와 둘레 계산 프로그램
2 #반지름 할당
3 rad=input("반지름을 입력하세요:")
4 #넓이 출력
5 print(f"원의 넓이는 {rad**2*3.141:.1f} 입니다.")
6 #둘레 출력
7 print(f"원의 둘레는 {2*rad*3.141:.1f} 입니다.")
```


복리 예금 계산기

- 원금(만 원 단위), 연이율(%), 예치 기간(년)을 입력받아 만기 시 받을 총 금액을 계산하시오.

$$\text{총금액} = \text{원금} \times (1 + \text{이율}/100) ** \text{기간}$$

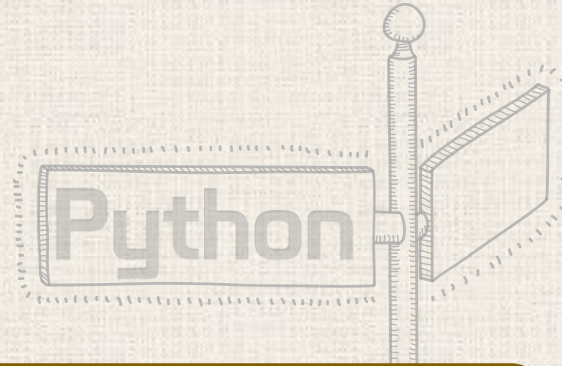
원금을 입력하세요(만 원): 100 ↵

이율을 입력하세요(%): 5 ↵

기간을 입력하세요(년): 10 ↵

10년 후 받을 금액은 약 162.89만 원입니다.

다음시간에는



4강. 순차 구조

➡ 5강. 선택 구조

6강. 반복 구조